

Bản sơ về ứng dụng hàm đánh giá trong thi tin học

Zhou Erjin

Trường Trung học số 1 Thiệu Hưng, Chiết Giang

2009

Nguồn gốc: Bản sơ về ứng dụng hàm đánh giá trong thi tin học

I0I China candidate team papers / 2009 / Zhou Erjin

Abstract

Hàm đánh giá có vai trò quan trọng trong một lớp bài toán tính chỉnh dân lời giải, nơi không gian trạng thái rất lớn và chuyển trạng thái tốn thời gian. Bài viết bàn về ứng dụng của hàm đánh giá trong thi tin học từ ba góc độ: tìm kiếm, tối ưu quy hoạch động và tìm lời giải khá tốt. Qua đó trình bày cách chọn, đánh giá và sử dụng hàm đánh giá để xử lý những bài vốn khó bắt đầu.

Từ khóa. Hàm đánh giá; tính chỉnh dân; tìm kiếm; quy hoạch động; lời giải khá tốt; cân bằng.

Contents

1	Mở đầu	2
2	Hàm đánh giá trong tìm kiếm	2
2.1	Bài toán tối ưu	2
2.1.1	Mines For Diamonds	2
2.1.2	Jaguar King	3
2.1.3	Tiểu kết	4
2.2	Bài toán khả thi	4
2.2.1	Sokoban	4
2.2.2	Tiểu kết	6
3	Hàm đánh giá trong tối ưu quy hoạch động	6
3.1	Largest Fence	6
3.2	Tiểu kết	7
4	Hàm đánh giá trong bài tìm lời giải khá tốt	7
4.1	Bài toán TSP	8
4.2	Tiểu kết	8

1 Mở đầu

Hàm đánh giá là một cách ánh xạ mức tốt xấu của trạng thái trong tương lai về một giá trị cụ thể. Ví dụ đơn giản là trong cờ vua: người mới chơi thường được cho biết tốt 1 điểm, tượng hoặc mã 3 điểm, xe 5 điểm, hậu 9 điểm. Với tiêu chuẩn đó, có thể xây dựng một đánh giá rất đơn giản: lấy giá trị quân có thể ăn ở nước kế tiếp làm giá trị đánh giá cho nước đó. Nếu đưa hàm đánh giá như vậy vào chương trình, chương trình vốn chỉ biết vết cạ đã có phần nào năng lực của người chơi nhập môn.

Đề thi tin học thường biến hóa đa dạng. Khi chưa tìm được phương pháp tối ưu, việc dùng hàm đánh giá hợp lý đôi khi đem lại hiệu quả rất bất ngờ.

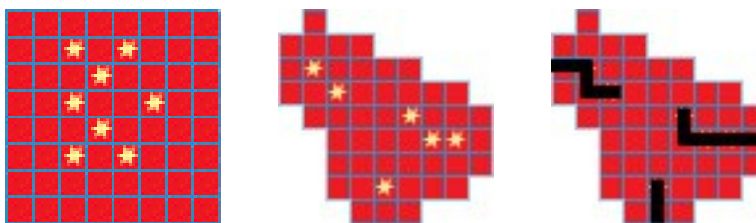
2 Hàm đánh giá trong tìm kiếm

2.1 Bài toán tối ưu

Trong thi tin học ta thường gặp các bài tối ưu mà bản thân bài toán khó có lời giải đa thức hiệu quả, nên phải dùng tìm kiếm. Không gian trạng thái của loại bài này thường rất lớn; vết cạ trực tiếp chắc chắn kém hiệu quả. Nếu sử dụng hàm đánh giá, ta có thể cắt nhánh đúng lúc và nâng cao hiệu suất chương trình.

2.1.1 Mines For Diamonds

Mô tả bài toán. Cho một mỏ kim cương kích thước $n \times m$. Cần đào một số đường hầm để lấy hết kim cương. Một đường hầm là một đường gấp khúc xuất phát từ biên; các viên kim cương nằm trên đường đều được lấy. Hai đường gấp khúc không được cắt nhau, nhưng có thể kề nhau. Hãy tìm tổng độ dài đường hầm nhỏ nhất. Giới hạn $n, m \leq 11$, số kim cương không quá 10.



Các hình nhỏ trong nguồn cho ví dụ Mines: bố trí kim cương và những đường hầm minh họa.

Phân tích. Vì phạm vi nhỏ và không thấy phương pháp đa thức rõ ràng, ta nghĩ tới tìm kiếm. Vết cạ trực tiếp sẽ TLE, nên cần cắt nhánh.

Khi độ sâu tăng, số nút sinh ra tăng cực nhanh. Nếu ở giai đoạn sớm có thể cắt bỏ một phần trạng thái thì hiệu quả sẽ tăng mạnh. Cách tự nhiên là dùng hàm đánh giá để cắt nhánh: nếu $g + h \geq ans$, không cần tiếp tục tìm kiếm. Ở đây g là chi phí đã dùng để tới trạng thái hiện tại, h là ước lượng lạc quan từ trạng thái hiện tại tới mục tiêu. Điều kiện cần là

$$h \leq h^*,$$

trong đó h^* là chi phí thật sự còn cần.

Nguồn đưa ra một dữ liệu mẫu và bảng số nút sinh bởi chương trình vết cạ:

độ sâu	1	2	3	4	5	6
số nút	9	41	145	515	1776	6120

độ sâu	7	8	9	10	11	12
số nút	20511	67935	220273	702437	2191682	6705638

độ sâu	13	14	15	16	17
số nút	20066463	58721500	167773970	467536227	120043037

Số liệu này cho thấy nếu không cắt nhánh sớm, cây tìm kiếm phình ra rất nhanh.

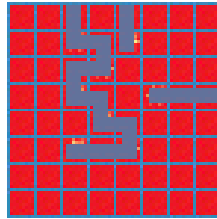
Vấn đề chính là thiết kế h . Quan sát đề, ràng buộc khó là hai đường hầm không được cắt nhau. Nếu bỏ qua ràng buộc này, bài toán chuyển thành: cho một tập điểm, chia chúng thành vài tập con; trong mỗi tập con nối các điểm bằng một dây chuyền sao cho tổng chi phí nhỏ nhất. Bài này có thể giải bằng quy hoạch động hai tầng. Gọi $\text{opt}[S]$ là lời giải tối ưu cho tập điểm S , ta có:

$$\text{opt}[S] = \min_{T \subset S} \{\text{opt}[S - T] + w[T]\}.$$

Ở đây $w[T]$ là chi phí nhỏ nhất để nối các điểm trong T bằng một dây chuyền:

$$w[T] = \min_{k \in T} \{w[T - \{k\}] + \text{dist}[T - \{k\}][k]\}.$$

Lấy $\text{opt}[S]$ làm hàm đánh giá vừa thỏa $h \leq h^*$, vừa thường rất gần chi phí tối ưu thực tế. Trong dữ liệu thử của bài gốc, số nút sinh ra giảm từ mức hàng chục triệu xuống rất nhỏ, và chương trình có thể AC trên UVa trong dưới 0.1 giây.



Dữ liệu dùng để minh họa hiệu quả của hàm đánh giá trong nguồn.

Với dữ liệu này, chương trình có hàm đánh giá sinh số nút như sau:

độ sâu	1	2	3	4	5	6
số nút	0	0	0	0	0	0

độ sâu	7	8	9	10	11	12
số nút	0	0	0	0	0	0

độ sâu	13	14	15	16	17
số nút	0	0	2112	43342	16733

Hiệu quả tối ưu vì thế rất rõ ràng.

2.1.2 Jaguar King

Mô tả bài toán. Có n con báo xếp hàng ở các vị trí $1..n$, n chia hết cho 4. Báo số 1 là vua báo; chỉ vua báo được đổi chỗ với báo khác. Nếu vua đang ở vị trí i , các nước nhảy hợp lệ phụ thuộc vào $i \bmod 4$:

$i \bmod 4$	vị trí có thể nhảy tới
1	$i + 1, i + 3, i - 4, i + 4$
2	$i + 1, i - 1, i - 4, i + 4$
3	$i + 1, i - 1, i - 4, i + 4$
0	$i - 3, i - 1, i - 4, i + 4$

Hỏi vua báo cần nhảy ít nhất bao nhiêu lần để xếp dãy đúng thứ tự. Giới hạn $n \leq 40$.

Phân tích. Bài toán giống mô hình 8-puzzle: chỉ một vị trí có thể di chuyển. Nếu sắp xếp các vị trí theo dạng bảng có 4 cột, ta thấy mỗi điểm thực chất có thể nhảy tới bốn ô kề. Vì thế có thể dùng đánh giá tương tự khoảng cách Manhattan:

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

$$E(s) = \sum_{i>1} \text{dist}(i, \text{pos}[i]),$$

trong đó $\text{dist}(i, j)$ là khoảng cách Manhattan trên bảng 4 cột:

$$x = \left| \left\lfloor \frac{i-1}{4} \right\rfloor - \left\lfloor \frac{j-1}{4} \right\rfloor \right|, \quad y = |(i-1) \bmod 4 - (j-1) \bmod 4|,$$

nếu $y = 3$ thì đặt $y = 1$, và $\text{dist}(i, j) = x + y$. Thêm đánh giá này vào tìm kiếm sâu lặp sẽ vượt qua dữ liệu UVa nhanh chóng.

Mở rộng. Nếu mỗi điểm V_i có một tập điểm tới được $(V_{i1}, \dots, V_{ik})$ với chi phí $(W_{i1}, \dots, W_{ik})$, không còn quan hệ Manhattan. Khi đó có thể dựng đồ thị $G = (V, E)$, với cạnh tương ứng và trọng số tương ứng, rồi dùng khoảng cách đường ngắn nhất:

$$E(s) = \sum_{i>1} \text{spp}[i][\text{pos}[i]].$$

Ở đây $\text{spp}[i][j]$ là đường ngắn nhất từ i tới j trong đồ thị G .

2.1.3 Tiểu kết

Với bài tối ưu dùng tìm kiếm, tìm kiếm mù thường duyệt quá nhiều trường hợp. Đánh giá kịp thời trạng thái trở thành yếu tố quan trọng. Hàm đánh giá hoàn hảo sẽ bằng đúng chi phí thật còn lại, nhưng khó có “lời tiên tri” như vậy. Trong thực tế, ta thường bỏ qua một ràng buộc nào đó của đề, biến bài toán thành một bài dễ hơn, rồi dùng lời giải của bài dễ hơn làm cận. Bài dễ hơn thường giải được bằng tham lam hoặc quy hoạch động, từ đó cắt bỏ rất nhiều trạng thái vô ích.

2.2 Bài toán khả thi

Có một lớp bài tìm kiếm không yêu cầu lời giải tối ưu, chỉ cần tìm một lời giải hợp lệ. Không gian trạng thái của chúng thường còn lớn hơn; cắt nhánh tối ưu kiểu thường dùng có thể làm hiệu quả giảm mạnh. Một cách hiệu quả là dùng hàm đánh giá để phân biệt trạng thái nào đáng thử trước.

2.2.1 Sokoban

Mô tả bài toán. Cho một thể Sokoban kích thước $n \times m$, cần tìm một lời giải không quá 10000 bước. Dữ liệu bảo đảm có lời giải, $n, m \leq 8$. Ký hiệu:

- #: tường;
- .: đích;
- @: vị trí ban đầu của người;
- +: người đứng trên đích;

- \$: hộp;
- *: hộp đứng trên đích.

```
#####
##   .#
#@ ###
#   * #
#   $ #
#   #
#####
```

Hai thể Sokoban minh họa trong nguồn.

Phân tích. Diện tích thực sự tối đa sau khi bỏ viền tường chỉ khoảng 6×6 , nên số hộp tối đa khoảng 35. Vì có nhiều hộp, để dễ viết ta xem mỗi bước là đẩy một hộp cho tới khi mọi hộp vào đích. Khó khăn lớn là deadlock: hộp bị đẩy vào góc tường, bốn hộp dính nhau, hộp sát một cạnh tường không có đích, v.v. Nhiều deadlock người nhìn ra ngay nhưng máy khó kiểm tra, còn kiểm tra thường xuyên lại tốn thời gian.

```
#####          #####          #####
#$   $#          #. . . #          # . #
#.   .#          # $ $ #          #   $#
#   @ #          # $ $ #          #.   #
#.   .#          #   #          # @ #
#$   $#          # @ #          # $ #
#####          #####          #####
```

Một số thể deadlock trong nguồn: góc tường, nhóm hộp kẹt nhau, và hộp sát tường không có đích tương ứng.

Thay vì đẩy hộp, ta suy nghĩ ngược: biến bài toán thành kéo hộp. Như vậy các deadlock nói trên tự nhiên biến mất.

Định nghĩa trạng thái $S(P, Q, pos)$, trong đó P là tập vị trí hộp, Q là tập đích, và pos là vị trí người. Dùng tìm kiếm sâu lặp: liệt kê giới hạn đáp án MAX . Nếu chi phí hiện tại là g , khi

$$g + h > MAX$$

thì cắt nhánh.

Một đánh giá hiển nhiên là khoảng cách Manhattan từ mỗi hộp tới một đích gần nhất:

$$h(S) = \sum_{i \in P} \min_{j \in Q} (|X_i - X_j| + |Y_i - Y_j|).$$

Đánh giá này hợp lệ nhưng chưa đủ mạnh; vì nó ghép nhiều hộp vào cùng một đích nên có thể đánh giá quá lạc quan. Cải tiến tự nhiên là dùng ghép cặp tối ưu KM giữa hộp và đích. Dù KM có độ phức tạp $\mathcal{O}(n^3)$, ở đây n nhỏ. Hơn nữa, sau một lần đẩy chỉ có một hộp đổi vị trí, nên từ ghép cặp tối ưu cũ, ghép cặp mới khác nhiều nhất hai cặp; có thể tối ưu cập nhật xuống gần $\mathcal{O}(n)$ sau lần đầu.

Nguồn thử 33 bộ dữ liệu với bước tăng độ sâu là 10. Dấu * chỉ bộ không ra nghiệm trong lần thử tương ứng; số trong ô là số nút đẩy hộp được sinh ra ở các vòng lặp sâu dần, không phải số bước lời giải cuối cùng.

test	Manhattan gần nhất	test	Manhattan gần nhất
0*	3, 3, 3, 597, 5360, 23115, ...	1	4, 4, 4, 185, 34
5*	4, 4, 1003, 10039, 32094, 51369, ...	7	6, 39, 2712, 11309, 2728
10*	4, 4, 2275, 40402, 120366, ...	16*	5, 5, 313, 9927, 64003, 49533, ...
21*	7, 12, 10828, 78396, ...	30*	4, 4, 354, 12355, 44301, 105225, ...

Sau khi thay đánh giá bằng KM, nhiều điểm trước đó không chạy ra nghiệm đã cải thiện rõ:

test	0*	1	2	3*	4	5	6	7	8	9	10*
nút	53838	34	69	53838	35	20421	514	4788	598	78	226089

test	11	12	13	14	15	16*	17	18	19	20	21
nút	20	135	132	4511	59	67341	15502	67	223	1023	5556

test	22	23	24	25	26*	27	28	29	30*	31	32
nút	1305	94	755	2828	70160	8054	4463	74378	163117	542	23859

Hàm đánh giá không chỉ dùng để cắt nhánh. Nó còn so sánh độ hứa hẹn giữa các con của một trạng thái. Nếu trước khi mở rộng một nút, ta sắp xếp các con theo hàm đánh giá và tìm trạng thái hứa hẹn trước, chương trình sẽ tìm lời giải sớm hơn nhiều. Trong bài gốc, sau tối ưu này mọi bộ thử đều chạy được và qua toàn bộ dữ liệu Ural.

Biểu đồ thời gian trong nguồn so sánh ba chương trình: A dùng khoảng cách Manhattan gần nhất, B dùng Manhattan với KM, C thêm sắp xếp con theo đánh giá trên nền B. Trục hoành là số hiệu dữ liệu, trục tung là thời gian theo mili giây. Đường của C thấp và ổn định nhất, cho thấy hàm đánh giá vừa dùng để cắt nhánh vừa dùng để chọn thứ tự mở rộng.

Lưu ý về tính đơn điệu. Hàm đánh giá trong Sokoban không đơn điệu, nghĩa là không bảo đảm lần đầu gặp một trạng thái là đường ngắn nhất tới trạng thái đó. Với tìm kiếm sâu lặp và bảng băm, điều này có thể làm mất một lời giải ở độ sâu hiện tại. Tuy nhiên bài chỉ yêu cầu một lời giải khả thi; nếu ở tầng hiện tại bỏ lỡ, ở tầng sâu hơn vẫn có thể tìm lại. Đổi lại, hàm đánh giá giúp hiệu quả tăng rất mạnh.

2.2.2 Tiểu kết

Với bài khả thi, hàm đánh giá phải thể hiện tốt sự khác biệt giữa các trạng thái. Nó không chỉ cho biết trạng thái hiện tại có đáng tiếp tục hay không, mà còn giúp so sánh hai trạng thái để nhanh chóng tìm được một lời giải. Tính không đơn điệu của hàm đánh giá trong loại bài này thường có thể chấp nhận được nếu đổi lại hiệu suất tốt hơn.

3 Hàm đánh giá trong tối ưu quy hoạch động

Quy hoạch động thường nổi tiếng vì gọn và hiệu quả. Nhưng nếu số trạng thái quá lớn, quy hoạch động trực tiếp vẫn lực bất tòng tâm. Khi đó có thể dùng hàm đánh giá để giảm số trạng thái hoặc ưu tiên thứ tự xử lý.

3.1 Largest Fence

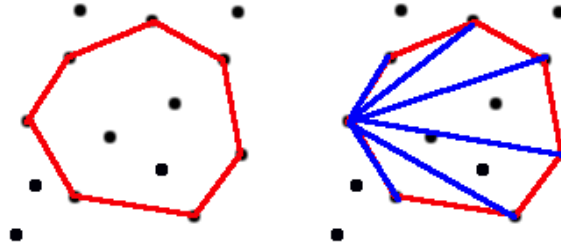
Mô tả bài toán. Farmer John có n điểm hàng rào, với $5 \leq n \leq 250$. Ông cần dựng một vòng hàng rào là một bao lồi và muốn số điểm trên bao lồi là nhiều nhất.

Phân tích. Một lời giải $\mathcal{O}(N^4)$ khá trực tiếp là: liệt kê điểm trái nhất trên bao lồi cuối cùng, sắp mọi điểm bên phải theo góc cực, rồi dùng trạng thái $\text{opt}[i][j]$ biểu thị “bao lồi” hiện tại có hai điểm cuối là i, j . Chuyển trạng thái:

$$\text{opt}[i][j] = \max\{\text{opt}[j][k] + 1\},$$

trong đó $\text{Point}(i)$ nằm bên trái đường thẳng $\text{Line}(k, j)$.

Số trạng thái là $\mathcal{O}(N^3)$, mỗi chuyển tốn $\mathcal{O}(N)$, nên tổng là $\mathcal{O}(N^4)$. Cách này chưa đủ nhanh cho dữ liệu lớn.



Hình nguồn: bao lồi cần tối đa hóa và quan sát dãy khoảng cách từ một đỉnh trên bao lồi tới các điểm còn lại.

Tối ưu 1. Nếu thiết kế được một hàm đánh giá $h(s)$ để phán đoán trạng thái hiện tại có thể dẫn tới nghiệm tốt hơn hay không, ta sẽ giảm được nhiều trạng thái. Quan sát một bao lồi, từ một đỉnh trên bao lồi tới các đỉnh khác, dãy khoảng cách thường có dạng đơn đỉnh: trước tăng rồi giảm. Điều này gợi ý bài toán dãy không giảm rồi không tăng.

Gọi $down[i]$ là độ dài dãy giảm dài nhất bắt đầu từ điểm i , và $up[i]$ là độ dài dãy đơn đỉnh dài nhất bắt đầu từ i :

$$down[i] = \max\{down[j] + 1 \mid j > i, \text{dist}[j] < \text{dist}[i]\},$$

$$up[i] = \max(down[i], \max\{up[j] + 1 \mid j > i, \text{dist}[j] > \text{dist}[i]\}).$$

Đặt

$$h[i][j] = up[j].$$

Nếu

$$opt[i][j] + h[i][j] \leq \text{ANS},$$

thì trạng thái này không thể dẫn tới đáp án tốt hơn và không cần mở rộng. Trong dữ liệu cực hạn $N = 250$, bước này giảm thời gian từ hơn 10 giây xuống khoảng vài giây.

Tối ưu 2. Giống bài Sokoban, hàm đánh giá còn có thể so sánh độ tốt của các trạng thái. Trong cùng một tầng chuyển, ưu tiên xử lý trạng thái con có đánh giá cao hơn. Làm vậy giúp tìm nghiệm tốt sớm hơn; nghiệm hiện tại càng tốt thì cắt nhánh càng mạnh. Sau tối ưu này, chương trình có thể giải dữ liệu cực hạn dưới một giây và vượt qua dữ liệu USACO.

3.2 Tiểu kết

Quy hoạch động rất hiệu quả, nhưng khi số trạng thái khổng lồ và chuyển trạng thái dày đặc, dùng trực tiếp thường khó đạt yêu cầu. Với các bài có dạng nhớ hóa tìm kiếm hoặc trạng thái có thể đánh giá, hàm đánh giá giúp bỏ qua trạng thái có triển vọng thấp hoặc ưu tiên trạng thái có triển vọng cao, nhờ đó nâng cao hiệu suất đáng kể.

4 Hàm đánh giá trong bài tìm lời giải khá tốt

Một số bài trong thi đấu chỉ yêu cầu lời giải khá tốt, không cần tối ưu tuyệt đối. Loại bài này linh hoạt hơn và kiểm tra năng lực phán đoán của thí sinh. Chúng thường là bài NP-đầy đủ hoặc khó tìm lời giải đa thức trong thời gian thi. Trong thực tế, dùng hàm đánh giá thường cho một thuật toán gọn và hiệu quả.

4.1 Bài toán TSP

Mô tả bài toán. Bài toán người bán hàng: cho n điểm trên mặt phẳng, tìm một chu trình đi qua mỗi điểm đúng một lần và có tổng độ dài nhỏ nhất. Đây là bài NP-đầy đủ kinh điển; với đồ thị tổng quát, việc tìm nghiệm tối ưu rất khó. Trong ứng dụng, thường chỉ cần một nghiệm khá tốt.

Phân tích. Đối với một trạng thái s , một đánh giá khả thi cần thỏa

$$h(s) \leq h^*(s).$$

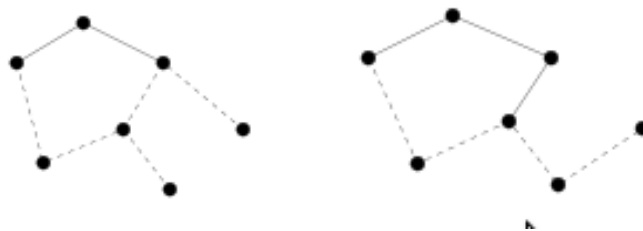
Ta lại nói lỏng ràng buộc: đề yêu cầu một chu trình Hamilton, mỗi điểm có bậc đúng bằng 2. Nếu bỏ qua ràng buộc đó, bài toán trở thành bài toán cây khung nhỏ nhất. Cụ thể, gọi tập điểm là P , và giả sử đã có một phần đường đi

$$Q = \{p_1, p_2, \dots, p_k\}.$$

Ta lấy tập điểm

$$P - Q + \{p_1, p_k\}$$

và tính cây khung nhỏ nhất trên tập này. Chi phí của cây đó là cận dưới cho phần đường còn lại.



Đường nét liền là phần đường đã chọn; đường nét đứt là cây khung nhỏ nhất dùng làm đánh giá cho phần còn lại.

Trong thử nghiệm của tác giả trên các bộ dữ liệu 50..100 điểm, chương trình dùng đánh giá MST cho nghiệm tốt hơn rõ rệt so với điều chỉnh ngẫu nhiên. Điều chỉnh ngẫu nhiên dễ rơi vào tối ưu cục bộ; dù có thể thêm biến dị kiểu thuật toán di truyền, hiệu quả vẫn không ổn định bằng cách dùng hàm đánh giá. Biểu đồ so sánh trong nguồn vẽ ba đường: nghiệm tối ưu, nghiệm dùng đánh giá MST và nghiệm dùng điều chỉnh ngẫu nhiên. Đường MST bám sát nghiệm tối ưu hơn rõ rệt trên hầu hết các bộ dữ liệu, còn điều chỉnh ngẫu nhiên dao động mạnh hơn do dễ kẹt ở tối ưu cục bộ.

4.2 Tiểu kết

Với bài chỉ cần nghiệm khá tốt, đề thường là NP-đầy đủ hoặc khó nghĩ ra lời giải đa thức trong thời gian thi. Nếu trạng thái của bài tương đối rõ, ta có thể chuyển bài gốc thành một bài dễ hơn để lấy cận trên hoặc cận dưới. Hàm đánh giá trong lớp bài này có thể linh hoạt hơn, và cần cân bằng giữa thời gian chạy với chất lượng nghiệm.

Tài liệu tham khảo

1. Frank Takes, *Sokoban: Reversed Solving*.
2. George F. Luger, *Artificial Intelligence: Structures and Strategies for Complex Problem Solving*.
3. Anand Panangadan, *Incremental Evaluation of Minimum Spanning Trees for the Traveling Salesman Problem*.

Lời cảm ơn

Tác giả cảm ơn các thầy cô đã bồi dưỡng và hướng dẫn; cảm ơn các bạn cùng tập luyện; và cảm ơn những người bạn trên mạng đã cùng trao đổi, thảo luận.